Mini Project 1

Big Data

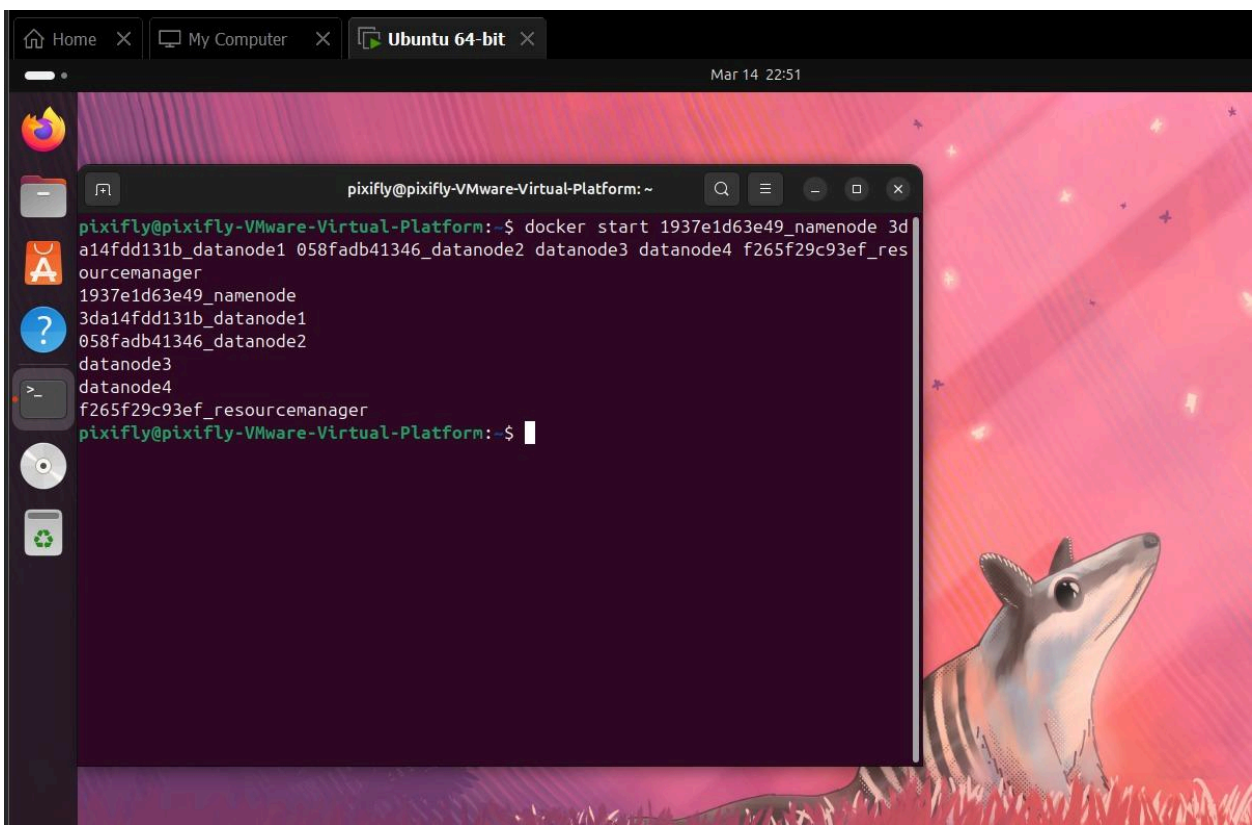
Riwan Ashraf 202201726
Rana Saad 202200902

1. Problem Statement

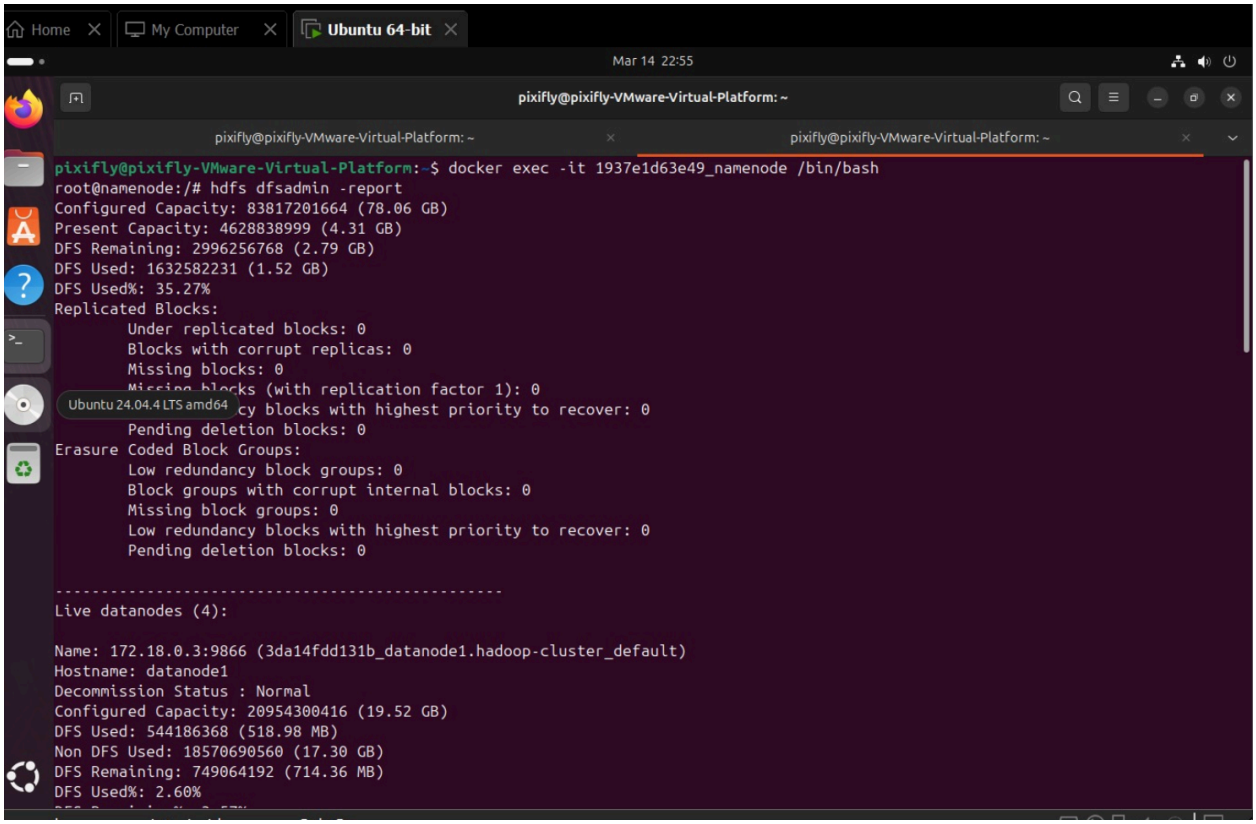
Sequentially scanning documents is slow for big libraries. As the size of
a digital library grows from megabytes to terabytes, the latency for simple keyword searches
We will be building a Reverse Index. This data structure is fundamental to search engines. Instead of mapping a document to the words it contains, a reverse index maps a word to a list of the documents it appears in, along with its frequency in that document.

2. Data Collection & Storage

We used classic books from Project Gutenberg, plus one custom document stored in HDFS at /user/student/library/. The books included Jane Eyre, The Count of Monte Cristo, Peter Pan, Sherlock Holmes, Tom Sawyer, A Tale of Two Cities, War and Peace, and Alice in Wonderland.HDFS was chosen because it splits files into 128 MB blocks, replicates them 3 times for fault tolerance .



docker start: namenode, datanode from 1 to 4, and resourcemanager are all confirmed running.



hdfs dfsadmin -report ; 4 live DataNodes

3. Preprocessing

We cleaned the text before emitting key-value pairs using the following steps:

- Lowercasing: line.lower() converts all words to lowercase.
- Remove punctuation: removes punctuation marks.
- Tokenization: line.split() splits the line into words.
- Stop-word removal: common words (like the, is, a, of, in,...etc) are removed using a stopwords.txt file loaded at the start. stop words make up about 40-60% of English text, so removing them reduces the number of intermediate outputs and decreases shuffle network traffic.

3 MapReduce

- Mapper: outputs (word, document.txt) for each valid word.
- Shuffle & Sort: Hadoop groups all values for the same word and sends them to the same Reducer.
- Reducer: counts occurrences and produces the final reverse index

```
root@namenode:/# hdfs dfs -ls /user/student/output_2nodes
Found 2 items
-rw-r--r--   3 root supergroup          0 2026-03-14 21:30 /user/student/output_2nodes/_SUCCESS
-rw-r--r--   3 root supergroup    2672828 2026-03-14 21:30 /user/student/output_2nodes/part-00000
```

Output from the 2-node cluster

```
root@namenode:/# hdfs dfs -cat /user/student/output_4nodes_final/part* | grep -i "happy"
2026-03-14 22:15:56,861 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTrusted = false, remoteHostTruste
d = false
approacheshappy --> tom_sawyer.txt:1
happy --> jane_eyre.txt:53, monte_cristo.txt:130, my_custom_index.txt:1, peter_pan.txt:12, sherlock.txt:18, tom_sawyer.txt:16, tw
o_cities.txt:29, war_and_peace.txt:194
happyalicetxt --> mapped_alice.txt:1
happygood --> my_custom_index.txt:1
happyhis --> monte_cristo.txt:1
happylooking --> my_custom_index.txt:1
happyto --> jane_eyre.txt:1
unhappy --> jane_eyre.txt:5, monte_cristo.txt:55, my_custom_index.txt:1, peter_pan.txt:8, sherlock.txt:4, tom_sawyer.txt:2, two_c
ities.txt:11, war_and_peace.txt:23
unhappyalicetxt --> mapped_alice.txt:2
unhappyvery --> jane_eyre.txt:1
```

Running grep "happy" on the 4-node cluster output shows that the word appears in different books such as war and peace ,monte cristo and jane eyre

3. Scalability Analysis

The same Hadoop Streaming job was executed four times using different cluster sizes. We used 4-node cluster as the baseline. Then DataNode containers were stopped one by one using docker stop to create 3-node, 2-node and 1-node configurations.

The execution time was measured using the Linux time command

```
real    0m32.165s
user    0m41.235s
sys     0m5.912s
```

4-node baseline, real: 32.165s

```
real    0m28.511s
user    0m39.303s
sys     0m5.074s
```

3-node, real: 28.511s

```
real    0m29.168s
user    0m40.989s
sys     0m4.819s
```

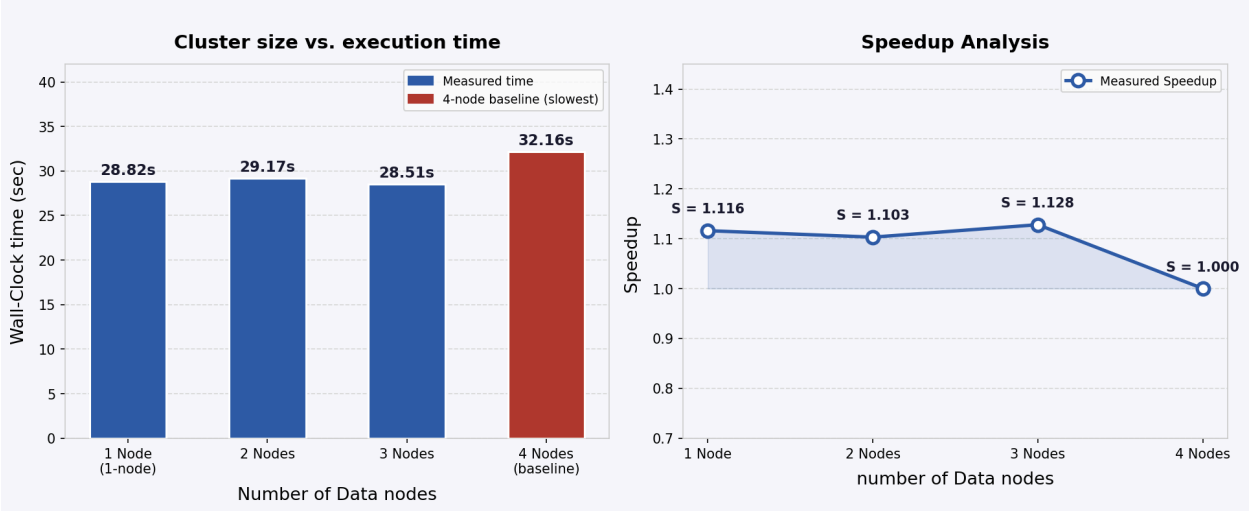
2-node, real: 29.168s

```
real    0m28.825s
user    0m41.015s
sys     0m4.635s
```

1-node, real: 28.825s

Configuration	Real time	User time	Sys time	Speedup
4 nodes	32.165	41.235	5.912	1
3 nodes	28.511	39.303	5.074	1.128
2 nodes	29.168	40.989	4.819	1.102
1 node	28.825	41.015	4.635	1.116

Speedup = 4-node time/node time



The results show that the 4-node configuration was the slowest while smaller clusters were slightly faster. This may seem unexpected but it can be explained by several reasons that may lead to that. First, the dataset (about 1.52 GB) is relatively small for Hadoop, so system overhead takes a large part of the runtime. Adding more nodes increases coordination overhead without significantly reducing computation time.

Overall, the project implemented a Distributed Reverse Index using Hadoop Streaming. The system preprocesses text, removes stop words, and generates a correct word-to-document frequency index. Testing with sample words confirmed that the index correctly mapped the word to multiple books with accurate counts. Although the scalability improvement was limited for this small dataset, the experiment shows that Hadoop MapReduce can scale effectively for much larger datasets and multi-machine clusters, making it suitable for large-scale text indexing.